

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.edu.cn
开始日期	2024.10.11	完成日期	2024.10.11

第一题

1. 实验题目

Huffman coding

2. 实验目的

In computer science and information theory, a Huffman code is an optimal prefix code algorithm.

In this exercise, please use Huffman coding to encode a given data.

You should output the number of bits, denoted as $B(T)$, to encode the data:

$$B(T) = \sum f(c) dT(c),$$

where $f(c)$ is the frequency of character c , and $dT(c)$ is be the depth of character c 's leaf in the tree T .

要求使用堆或优先队列来完成, 会检查代码和报告 (报告中请附上代码)。

3.算法设计

- 1.定义类和 vector 数组存储输入的数据，内容 word 和频率 fre
- 2.定义哈夫曼树结点结构体，左子树 left，右子树 right，内容 data，频率 fre
- 3.使用优先队列构造哈夫曼树，定义优先队列和自定义类型排序的比较重载函数
- 4.将输入的内容放入优先队列进行排序
- 5.构造哈夫曼树:①取出优先队列的前两个结点 ②将这两个结点组成新结点 ③将新结点重新放入优先队列 ④重复上述操作直到优先队列的大小为 1
- 6.计算 B (T) 函数，对哈夫曼树进行遍历，遍历到叶子结点时进行计算，同时每往下一层就深度加一，回溯保持同层次深度不变

4.程序运行与测试

- 1.定义优先队列及其自定义类型排序的重载函数

```
// 比较器，用于优先队列中的节点排序
struct m_compare
{
    bool operator()(TreeNode* & t1,TreeNode* & t2)
    {
        // 优先比较频率，如果频率相同，再按字母顺序比较
        if (t1->fre == t2->fre)
            return t1->data > t2->data; // 字母顺序按升序排列
        else
            return t1->fre > t2->fre; // 按频率升序排列
    }
};

// 使用优先队列来构建哈夫曼树
priority_queue<TreeNode*, vector<TreeNode*>, m_compare> pq;
```

- 2.定义存储输入的自定义类型和定义哈夫曼树的结点类型

```

// 定义字符结构体，包含字符和频率
class Word
{
public:
    char word; // 字符
    int fre;    // 频率
    Word(char w, int f) : word(w), fre(f) {}
};

// 定义哈夫曼树节点结构体
struct TreeNode
{
    TreeNode* left;
    TreeNode* right;
    char data;
    int fre;

    // 构造函数
    TreeNode(char d, int f, TreeNode* l = nullptr, TreeNode* r = nullptr)
        : data(d), fre(f), left(l), right(r) {}
};

```

3.核心部分：哈夫曼树的构建

```

// 构建哈夫曼树
while (pq.size() > 1)
{
    // 取出最小的两个节点
    TreeNode* t1 = pq.top(); pq.pop();
    TreeNode* t2 = pq.top(); pq.pop();

    // 创建新节点合并这两个节点
    TreeNode* newNode = new TreeNode(t2->data, t1->fre + t2->fre, t2, t1);

    // 将新的节点重新放入优先队列
    pq.push(newNode);
}

```

4.计算 B (T) 的函数

```

void CalTime(TreeNode* root,int depth,int& sum)
{
    if(root->left==NULL || root->right==NULL)//叶子结点
    {
        sum=sum+depth*(root->fre);
        return ;
    }
    CalTime(root->left,depth+1,sum);
    CalTime(root->right,depth+1,sum);
}

```

5.实验总结与心得

- 1.第一题和第三题的实现模板相似，区别在于第三题是最后要打印后序遍历的哈夫曼树
- 2.题目难度较大，需要设计好各个模块，重点包括优先队列、构建哈夫曼树的过程
- 3.优先队列的函数重载，要使得结果为升序排列，由小到大，就要返回 `t1->data>t2->data`，和一般的 `sort` 有些不同
- 4.此处优先队列存储的是自定义类型 `TreeNode`，平时使用较少，要注意其格式
- 5.优先队列排序若有问题时，可在调试时将排序结果打印在控制台上观察。

第二题

1. 实验题目

判断一个图是否构成树

2. 实验目的

问题

给定一个无向图，判断该图是否构成树。

输入

输入有若干测试样例。第一行是测试样例个数，接下来若干测试样例。

每个测试样例的第一行是结点数 n ，而且结点用 $1,2,...,n$ 编号。

第二行是边数 m ，接下来是 m 个结点对。

输出

如果一个图是树，则打印 YES，否则打印 NO。每个输出占一行。

3. 算法设计

- 1.重点在于考虑：如果一个图是树，则它必须是一个连通图，并且它的边数必须等于 $n - 1$
- 2.记录每一组数据的结点个数
- 3.记录每一组数据的边的个数
- 4.记录每一次输入边时是否涉及到了所有的结点，以保证所有结点是连在一起的

4. 程序运行与测试

- 1.代码较简单，主要是判断的逻辑：如果一个图是树，则它必须是一个连通图，并且它的边数必须等于 $n - 1$

```
int num;//结点数
cin>>num;
int edge;//边数
cin>>edge;
int nn=edge;
set<int> s;
while(nn-->0)
{
    int a,b;
    cin>>a>>b;
    s.insert(a);
    s.insert(b);
}
if(num==edge+1 && s.size()==num)
//如果一个图是树，则它必须是一个连通图，并且它的
//边数必须等于  $n - 1$ 
    cout<<"YES"<<endl;
else
    cout<<"NO"<<endl;
```

5. 实验总结与心得

- 1.题目较为简单，重要在于能否熟悉图论的知识，能否快速直到：如果一个图是树，则它必须是一个连通图，并且它的边数必须等于 $n - 1$
- 2.然后根据这一条件，对输入的边和结点进行判断即可

第三题

1. 实验题目

哈夫曼树编码

2. 实验目的

给定 K 个不同的字符和它们的频率，建立一棵哈夫曼树对字符进行编码，合并节点时频率小的在右边，频率相同时字符小的在右边。

按照后序遍历的顺序输出字符。

输入包含两部分，第一部分一个数字 K 表示有 K 个字符，第二部分有 K 行，每行两个数表示字符和它的频率

3. 算法设计

- 1.定义类和 vector 数组存储输入的数据，内容 word 和频率 fre
- 2.定义哈夫曼树结点结构体，左子树 left，右子树 right，内容 data，频率 fre
- 3.使用优先队列构造哈夫曼树，定义优先队列和自定义类型排序的比较重载函数
- 4.将输入的内容放入优先队列进行排序
- 5.构造哈夫曼树：①取出优先队列的前两个结点 ②将这两个结点组成新结点 ③将新结点重新放入优先队列 ④重复上述操作直到优先队列的大小为 1
- 6.对哈夫曼树进行后序遍历

4. 程序运行与测试

- 1.定义优先队列及其自定义类型排序的重载函数

```

// 比较器，用于优先队列中的节点排序
struct m_compare
{
    bool operator()(TreeNode* & t1,TreeNode* & t2)
    {
        // 优先比较频率，如果频率相同，再按字母顺序比较
        if (t1->fre == t2->fre)
            return t1->data > t2->data; // 字母顺序按升序排列
        else
            return t1->fre > t2->fre; // 按频率升序排列
    }
};

// 使用优先队列来构建哈夫曼树
priority_queue<TreeNode*, vector<TreeNode*>, m_compare> pq;

```

2.定义存储输入的自定义类型和定义哈夫曼树的结点类型

```

// 定义字符结构体，包含字符和频率
class Word
{
public:
    char word; // 字符
    int fre; // 频率
    Word(char w, int f) : word(w), fre(f) {}
};

// 定义哈夫曼树节点结构体
struct TreeNode
{
    TreeNode* left;
    TreeNode* right;
    char data;
    int fre;

    // 构造函数
    TreeNode(char d, int f, TreeNode* l = nullptr, TreeNode* r = nullptr)
        : data(d), fre(f), left(l), right(r) {}
};

```

3.核心部分：哈夫曼树的构建

```

// 构建哈夫曼树
while (pq.size() > 1)
{
    // 取出最小的两个节点
    TreeNode* t1 = pq.top(); pq.pop();
    TreeNode* t2 = pq.top(); pq.pop();

    // 创建新节点合并这两个节点
    TreeNode* newNode = new TreeNode(t2->data, t1->fre + t2->fre, t2, t1);

    // 将新的节点重新放入优先队列
    pq.push(newNode);
}

```

5. 对哈夫曼树进行后序遍历输出

```

// 后序遍历输出哈夫曼树
void PostOrderTraversal(TreeNode* root)
{
    if (root)
    {
        PostOrderTraversal(root->left); // 遍历左子树
        PostOrderTraversal(root->right); // 遍历右子树
        if (root->left == NULL || root->right == NULL) // 如果是叶子节点，输出字符
            cout << root->data << endl;
    }
}

```

5.实验总结与心得

1.第三题和第一题内容实现模板相同

2.解决该题时遇到难点之一是排序，按照题目要求合并节点时频率小的在右边，频率相同时字符小的在右边，因此设计函数重载时要注意判断条件

// 优先比较频率，如果频率相同，再按字母顺序比较

if (t1->fre == t2->fre)

return t1->data > t2->data; // 字母顺序按升序排列

else

return t1->fre > t2->fre; // 按频率升序排列

3.在构建哈夫曼树时，要将先取出来的 t1 放在右边，后取出来的 t2 放在左边

4.之后，不仅需要将根节点的 fre 设为左右子树 fre 之和，还需要将根节点的 data 设为左右字符中较大者即 t2 的 data，以便后续子树再被拿出来作比较。

// 创建新节点合并这两个节点

TreeNode* newNode = new TreeNode(t2->data, t1->fre + t2->fre, t2, t1);